

Issue I: Martin-Löf Type Theory

Maksym Sokhatskyi

National Technical University of Ukraine

Igor Sikorsky Kyiv Polytechnical Institute

June 11, 2026

Abstract

Martin-Löf Type Theory (MLTT), introduced by Per Martin-Löf in 1972, is a cornerstone of constructive mathematics, providing a foundation for formalizing mathematical proofs and programming languages. Its 1973 variant, MLTT-73, incorporates dependent types (Π , Σ) and identity types (Id), with the J eliminator as a key construct for reasoning about equality. Historically, internalizing MLTT in a type checker while constructively proving the J eliminator has been challenging due to limitations in pure functional systems. This article presents a canonical formalization of MLTT-73 and its internalization (without η -rule for identity types due to groupoid interpretation) in **Per**, a dependent type theory language equipped with cubical type primitives. Using presented type theory, we constructively prove induction and computation MLTT-73 inference rules, including the J eliminator, and demonstrate suitability as a robust foundation for mathematical languages.

Keywords: Martin-Löf Type Theory, Cubical Type Theory.

Contents

1	Interpretations	3
1.1	Type Theory	3
1.2	Logic	3
1.3	Category Theory	4
1.4	Homotopy Theory	4
2	Dependent Type Theory	5
2.1	Universes (U_i)	5
2.2	Dependent Product (Π)	6
2.3	Dependent Sum (Σ)	10
2.4	Path Space (Ξ)	12

Introduction to MLTT

For decades, type theorists have sought to fully internalize Martin-Löf Type Theory (MLTT) within a type checker, a task akin to building a self-verifying blueprint for mathematics.

Introduced by Per Martin-Löf in 1972 [2] MLTT-72 had only Π and Σ types. In 1973, a variant MLTT-73 with Id types was introduced with countable hierarchy of universes. In 1975, a variant of MLTT-75 with Π and Σ , Id, $+$, and $\mathbb{N}$ type was officially introduced [3] including infinite predicative hierarchy of universes.

Central to MLTT-73 is the J eliminator, a rule that governs how identity proofs are used, but its constructive derivation has long eluded pure functional type checkers due to the complexity of equality types. This article addresses this challenge by presenting a canonical formalization of MLTT-73 and its internalization in **Per**, a novel type theory language designed for constructive proofs.

Leveraging cubical type theory [14], this language incorporates Path types and universe polymorphism to faithfully embed MLTT-73 rules, achieving a constructive proof of the J eliminator. This internalization serves as an ultimate test of a type checker’s robustness, verifying its ability to fuse and full coverage of introduction and elimination rules through beta and eta equalities.

To make MLTT accessible, we provide intuitive interpretations of its types: logical (as quantifiers), categorical (as functors), and homotopical (as spaces). These perspectives highlight MLTT’s role as a bridge between mathematics and computation. Our work builds on Martin-Löf’s vision of constructive mathematics, offering a minimal yet powerful framework for mechanized reasoning. We aim to inspire researchers and practitioners to explore type theory’s potential in formalizing mathematics and designing reliable software.

Syntax of Per

The BNF consists of: i) telescopes (contexts) and definitions; ii) pure dependent type theory syntax; iii) identity system; iv) cubical face system; v) module system. It is slightly based on cubicaltt.

```

F = module I where L
L =  $\emptyset$  | import I | def I T : O := O
T =  $\emptyset$  | ( I : O ) T
O = I | U | ( O )
    |  $\Pi$  ( I : O ) , O |  $\lambda$  ( I : O ) , O | O  $\rightarrow$  O | O O
    |  $\Xi$  O O O O O |  $\langle$  O  $\rangle$  O | O @ O | transp O O
    | 0 | 1 |  $\neg$ O | O  $\wedge$  O | O  $\vee$  O |  $\square$ 
    |  $\Sigma$  ( I : O ) , O | O .1 | O .2 | O , O

```

Here, $=$ (definition), $\emptyset$ (empty set), $|$ (vertical bar) — are parts of BNF language and $\langle, \rangle$, $(,)$, $:=$, $\vee$, $\wedge$, $\neg$, $\rightarrow$, 0, 1, @, $\square$, **module**, **import**, **where**, **transp**, .1, .2, and , are terminals of the type checker language ¹.

¹<https://github.com/groupoid/per>

1 Interpretations

1.1 Type Theory

In MLTT, types are defined by five classes of rules: (1) *formation*, specifying the type’s signature; (2) *introduction*, defining constructors for its elements; (3) *elimination*, providing a dependent induction principle; (4) *computation* (beta-equality), governing reduction; and (5) *uniqueness* (eta-equality), ensuring canonical forms, though the latter is absent for identity types in homotopical settings.

For MLTT-73, we focus on Π (dependent function types), Σ (dependent pair types), and Id (identity types). MLTT-72 provided the foundational Π and Σ types, lacking mechanisms for equality, which MLTT-73 introduced via Id types, originally assuming uniqueness of identity proofs (UIP) in some contexts [3]. In cubical type theory, Id types are replaced by **Path** types, defined as functions from an interval $[0, 1]$, making the J eliminator computationally effective and supporting constructive proofs [1]. The identity type, introduced in MLTT-73 and refined in [3], is significant for enabling constructive equality reasoning.

Modern homotopical interpretations, pioneered by Hofmann and Streicher [6], refute UIP, adopting **Path** types that model equality as paths in a space, aligning with cubical type theory’s constructive framework [14]. This shift, integral to MLTT-75, facilitates the internalization of MLTT-73 rules.

Type checkers operate within contexts, binding variables to indexed universes, built-in types, or user-defined types via de Bruijn indices (to avoid variable capture) or names (for user-friendly proof assistants). These contexts, central to MLTT implementations, enable queries about type derivability and code extraction, forming the core of type checkers. As shown in Table 1 MLTT-75 unifies these constructs across multiple domains.

1.2 Logic

The logical interpretation casts MLTT-75 as a system for intuitionistic higher-order logic, where types correspond to propositions and terms to proofs, embodying the Curry-Howard correspondence. In this view, a type A represents a proposition, and a term $a : A$ is a proof of A . The Π -type, $\prod_{x:A} B(x)$, encodes universal quantification ($\forall x : A, B(x)$), while the Σ -type, $\sum_{x:A} B(x)$, represents existential quantification ($\exists x : A, B(x)$). The identity type, $\text{Id}_A(a, b)$, captures propositional equality ($a =_A b$), with the J eliminator providing a constructive means to reason about equalities.

Each type’s five rules (formation, introduction, elimination, computation, and uniqueness, except for Id in cubical settings) mirror the structure of logical inference rules. For instance, the introduction rule for Π constructs a lambda term (proof of a universal statement), while its elimination rule applies the term to an argument (using the universal statement).

MLTT-73 is not standalone framework for constructive mathematics but rather the extended foundational core on top of MLTT-72. Adding **0** (Empty),

Type Theory	Logic	Category Theory	Homotopy Theory
A type	class	object	space
$\text{isProp } A$	proposition	(-1) -truncated object	space
$a:A$ program	proof	generalized element	point
$B(x)$	predicate	indexed object	fibration
$b(x) : B(x)$	conditional proof	indexed elements	section
$\mathbf{0}$	$\perp$ false	initial object	empty space
$\mathbf{1}$	$\top$ true	terminal object	singleton
$\mathbf{2}$	boolean	subobject classifier	$\mathbb{S}^0$
$A + B$	$A \vee B$ disjunction	coproduct	coproduct space
$A \times B$	$A \wedge B$ conjunction	product	product space
$A \rightarrow B$	$A \Rightarrow B$	internal hom	function space
$\sum x : A, B(x)$	$\exists_{x:A} B(x)$	dependent sum	total space
$\prod x : A, B(x)$	$\forall_{x:A} B(x)$	dependent product	space of sections
Path_A	equivalence $=_A$	path space object	path space A^I
quotient	equivalence class	quotient	quotient
W-type	induction	colimit	complex
type of types	universe	object classifier	universe
quantum circuit	proof net	string diagram	

$\mathbf{1}$ (Unit) types allows resulting type system to internalize intuitionistic propositional logic (IPL), extending further with $\mathbf{2}$ (Bool) it can encode classical logic with the rule of excluded middle (CPL) [10].

1.3 Category Theory

The categorical interpretation models MLTT-75 within category theory, where types are objects, terms are morphisms, and type constructions are functors. This perspective, formalized by Cartmell and Seely [13], views MLTT-75 with $\mathbf{0}$, $\mathbf{1}$, $\mathbf{2}$ types as a locally cartesian closed category (LCCC) with boolean as subobject classifiers forming boolean topoi. Here, Π -types correspond to dependent products (right adjoints to base change functors), and Σ -types to dependent sums (left adjoints). The identity type, Id_A , is modeled as a path space object, reflecting equality as a morphism.

For example, given a morphism $f : A \rightarrow B$ in a category, the Π_f functor maps a dependent type over B to one over A , generalizing function spaces, while Σ_f constructs the total space of a fibration.

1.4 Homotopy Theory

The homotopical interpretation, a breakthrough in modern type theory, views MLTT-73 types as spaces and terms as points, with identity types as paths. Introduced by Hofmann and Streicher's groupoid model [6], this perspective refutes the uniqueness of identity proofs (UIP) in classical MLTT-73, replacing Id with Path types that model equality as continuous paths in a space. In

cubical type theory, Path types are functions from an interval $[0, 1]$ to a type, enabling constructive proofs of MLTT-73 rules, including the J eliminator.

Here, Π -types represent spaces of sections, Σ -types denote total spaces of fibrations, and Path types form path spaces (A^I) . This interpretation connects MLTT-73 to homotopy theory, where types are ∞ -groupoids, and fibrations (dependent types) are studied geometrically. For instance, a Π -type can be seen as a trivial fiber bundle, with its introduction rule constructing a section [1].

Set Theory

The set-theoretical interpretation models MLTT-75's types as sets and terms as elements, aligning with classical first-order logic. In this view, a type A is a set, and a term $a : A$ is an element. The Π -type represents a set of functions, Σ -type a disjoint union of sets, and $\text{Id}_A(a, b)$ an equality relation. However, this interpretation is limited, as it cannot capture higher equalities (e.g., paths between paths) or inductive types directly, due to its 0-truncated nature [1].

2 Dependent Type Theory

2.1 Universes (U_i)

In Martin-Löf Type Theory (MLTT), universes are types that classify other types, forming a cumulative hierarchy to manage type formation and avoid paradoxes like Russell's. MLTT-73 adopts a predicative hierarchy of universes, denoted U_i for $i \in \mathbb{N}$, where each universe U_i is a type in the next universe U_{i+1} .

This section defines the universe hierarchy constructively, specifying formation, introduction, and computation rules, and illustrates their encoding in **Per**.

Definition 1 (Universe Formation). For each natural number $i \in \mathbb{N}$, there exists a universe U_i , which is a type classifying small types at level i . The formation rule is: $\Gamma \vdash U_i : U_{i+1}$. Universes are introduced as constructors, with each U_i inhabiting U_{i+1} .

```
def U (i : Nat) : U (suc i)
```

Definition 2 (Universe Introduction). A type A belongs to a universe U_i if it can be derived as a type at level i . For MLTT-73, this includes base types (e.g., Π , Σ , Path), user-defined types, and universes U_j for $j < i$. The introduction rule is: $\Gamma; A \vdash A : U_i$, where i is the minimal level such that $A \in U_i$. Types like $\Pi(A, B)$, $\Sigma(A, B)$, and $\Xi(A, x, y)$ are explicitly landed in a universe:

```
def Pi (A : U i) (B : A → U i) : U i := Π (x : A), B x
def Sigma (A : U i) (B : A → U i) : U i := Σ (x : A), B x
def Path (A : U i) (x y : A) : U i := PathP (<-> A) x y
```

Definition 3 (Cumulative Hierarchy). The universe hierarchy is cumulative, meaning if $A : U_i$, then $A : U_j$ for all $j > i$. This ensures flexibility in type checking, as types can be lifted to higher universes. This is implicit in the type checker's ability to assign types to higher universes when needed.

Definition 4 (Predicative Rules). The formation of dependent types (e.g., Π , Σ) lands in the maximum of the universe levels of its constituents. For example, for Π -types: $\Gamma \vdash A : U_i$ and $\Gamma, x : A \vdash B(x) : U_j$ we can derive $\Gamma \vdash \Pi(x : A), B(x) : U_{\max(i,j)}$. This predicative rule ensures that the universe level reflects the highest level of the domain or codomain.

```
def Level (i j : N) (A : U i) (B : A -> U j)
  : U (max i j) := Pi (x : A), B x
```

Similar rules apply to Σ and Path types, ensuring all MLTT-73 types are predicatively landed.

Definition 5 (Definitional Equality). Universes support definitional equality, where two types $A, B : U_i$ are equal if their normalized forms are identical. This is crucial for type checking in MLTT-73.

2.2 Dependent Product (Π)

Π is a dependent product type, the generalization of functions. As a function it can serve the wide range of mathematical constructions as its domain and codomain, which are in general: objects, types, or spaces; and could have as its instance: sets, functions, polynomial functors, infinitesimals, ∞ -groupoids, topological ∞ -groupoid, CW-complexes, categories, languages, etc.

At this light there could be many interpretation of Π types from different areas of mathematics. We give here three: i) logical interpretation of Π as $\forall$ quantifier from higher order logic that forms a ground of type theory; ii) geometric interpretation of Π as fiber bundle; iii) categorical interpretation of functions as functors.

Type-theoretical interpretation

As a logical system dependent type theory could correspond to higher order logic. However here only type-theoretical model is given completely.

Definition 6 (Π -Formation). Π -types represents the way we create the spaces of dependent functions $f : \Pi(x : A), B(x)$ with domain in A and codomain in type family $B : A \rightarrow U$ over A .

$$\Pi(A, B) : U =_{def} \prod_{A:U} \prod_{B:A \rightarrow U} \prod_{x:A} B(x).$$

```
def Pi (A : U) (B : A -> U) : U := Pi (x : A), B x
```

Definition 7 (Π -Introduction). Lambda constructor defines a new lambda function in the space of dependent functions. It is called lambda abstraction and displayed as $\lambda x.b(x)$ or $x \mapsto b(x)$.

$$\lambda(x : A), b(x) : \Pi(A, B) =_{def}$$

$$\prod_{A:U} \prod_{B:A \rightarrow U} \prod_{b:\Pi(A,B)} \lambda x, b_x.$$

```
def lambda (A: U) (B: A → U) (b: Pi A B) : Pi A B := λ (x : A), b x
def lam (A B: U) (f: A → B) : A → B := λ (x : A), f x
```

When codomain is not dependent on valude from domain the function $f : A \rightarrow B$ is studied in System F_ω , dependent case in studied in System P_ω or Calculus of Construction (CoC).

Definition 8 (II-Induction Principle). States that if predicate holds for lambda function then there is a function from function space to the space of predicate.

```
def II-ind (A : U) (B : A → U) (C : Pi A B → U) (g: Π (x: Pi A B), C x)
  : Π (p: Pi A B), C p := λ (p: Pi A B), g p
```

Definition 9 (II-Elimination). Application reduces the term by using recursive substitution.

$$f a : B(a) =_{def} \prod_{A:U} \prod_{B:A \rightarrow U} \prod_{a:A} \prod_{f:\prod_{x:A} B(a)} f(a).$$

```
def apply (A: U) (B: A → U) (f: Pi A B) (a: A) : B a := f a
def app (A B: U) (f: A → B) (x: A) : B := f x
```

Theorem 1 (II-Composition). Composition is using application of appropriate singnatures.

$$f(a) =_{B(a)} (\lambda(x : A) \rightarrow f(a))(a).$$

```
def o⊤ (α β γ: U) : U
:= (β → γ) → (α → β) → (α → γ)
```

```
def o (α β γ : U) : o⊤ α β γ
:= λ (g: β → γ) (f: α → β) (x: α), g (f x)
```

Theorem 2 (II-Computation). β -rule shows that composition $\text{lam} \circ \text{app}$ could be fused.

$$f(a) =_{B(a)} (\lambda(x : A) \rightarrow f(a))(a).$$

```
def II-β (A : U) (B : A → U) (a : A) (f : Pi A B)
  : Path (B a) (apply A B (lambda A B f) a) (f a)
:= idp (B a) (f a)
```

Theorem 3. (II-Uniqueness). η -rule shows that composition $\text{app} \circ \text{lam}$ could be fused.

$$f =_{(x:A) \rightarrow B(a)} (\lambda(y : A) \rightarrow f(y)).$$

```
def II-η (A : U) (B : A → U) (a : A) (f : Pi A B)
  : Path (Pi A B) f (λ (x : A), f x)
:= idp (Pi A B) f
```

Categorical interpretation

The adjoints Π and Σ is not the only adjoints could be presented in type system. Axiomatic cohesions could contain a set of adjoint pairs as a core type checker operations.

Definition 10 (Dependent Product). The dependent product along morphism $g : B \rightarrow A$ in category C is the right adjoint $\Pi_g : C_{/B} \rightarrow C_{/A}$ of the base change functor.

Definition 11 (Space of Sections). Let $\mathbf{H}$ be a $(\infty, 1)$ -topos, and let $E \rightarrow B : \mathbf{H}_{/B}$ a bundle in $\mathbf{H}$, object in the slice topos. Then the space of sections $\Gamma_\Sigma(E)$ of this bundle is the Dependent Product:

$$\Gamma_\Sigma(E) = \Pi_\Sigma(E) \in \mathbf{H}.$$

Theorem 4 (Homotopy Equivalence). If fiber space is set for all base, and there are two functions $f, g : (x : A) \rightarrow B(x)$ and two homotopies between them, then these homotopies are equal.

```
def setPi (A: U) (B: A → U)
  (h: Π (x: A), isSet (B x)) (f g: Pi A B)
  (p q: Path (Pi A B) f g)
  : Path (Path (Pi A B) f g) p q
```

Theorem 5 (Contractability). If domain and codomain is contractible then the space of sections is contractible.

```
def piIsContr (A: U) (B: A → U) (u: isContr A)
  (q: Π (x: A), isContr (B x))
  : isContr (Pi A B)
```

Definition 12 (Section). A section of morphism $f : A \rightarrow B$ in some category is the morphism $g : B \rightarrow A$ such that $f \circ g : B \xrightarrow{g} A \xrightarrow{f} B$ equals the identity morphism on B .

Homotopical interpretation

Geometrically, Π type is a space of sections, while the dependent codomain is a space of fibrations. Lambda functions are sections or points in these spaces, while the function result is a fibration. Π type also represents the cartesian family of sets, generalizing the cartesian product of sets.

Definition 13. (Fiber). The fiber of the map $p : E \rightarrow B$ in a point $y : B$ is all points $x : E$ such that $p(x) = y$.

Definition 14. (Fiber Bundle). The fiber bundle $F \rightarrow E \xrightarrow{p} B$ on a total space E with fiber layer F and base B is a structure (F, E, p, B) where $p : E \rightarrow B$ is a surjective map with following property: for any point $y : B$ exists a neighborhood U_b for which a homeomorphism $f : p^{-1}(U_b) \rightarrow U_b \times F$ making the following diagram commute.

$$\begin{array}{ccc}
p^{-1}(U_b) & \xrightarrow{f} & U_b \times F \\
p \downarrow & \swarrow pr_1 & \\
U_b & &
\end{array}$$

Definition 15. (Cartesian Product of Family over B). Is a set F of sections of the bundle with elimination map $app : F \times B \rightarrow E$ such that

$$F \times B \xrightarrow{app} E \xrightarrow{pr_1} B \quad (1)$$

pr_1 is a product projection, so pr_1, app are morphisms of slice category Set/B . The universal mapping property of F : for all A and morphism $A \times B \rightarrow E$ in Set/B exists unique map $A \rightarrow F$ such that everything commute. So a category with all dependent products is necessarily a category with all pullbacks.

Definition 16 (Trivial Fiber Bundle). When total space E is cartesian product $\Sigma(B, F)$ and $p = pr_1$ then such bundle is called trivial $(F, \Sigma(B, F), pr_1, B)$.

Theorem 6 (Functions Preserve Paths). For a function $f : (x : A) \rightarrow B(x)$ there is an $ap_f : x =_A y \rightarrow f(x) =_{B(x)} f(y)$. This is called application of f to path or congruence property (for non-dependent case — *cong* function). This property behaves functorially as if paths are groupoid morphisms and types are objects.

Theorem 7 (Trivial Fiber Bundle equals Family of Sets). Inverse image (fiber) of fiber bundle $(F, B * F, pr_1, B)$ in point $y : B$ equals $F(y)$.

```

def Family (B : U) : U1 := B → U
def Fibration (B : U) : U1 := Σ (X : U), X → B

def encode-Pi (B : U) (F : B → U) (y : B)
  : fiber (Sigma B F) B (pr1 B F) y → F y
:= λ (x : fiber (Sigma B F) B (pr1 B F) y),
    subst B F x.1.1 y (<i> x.2 @ -i) x.1.2

def decode-Pi (B : U) (F : B → U) (y : B)
  : F y → fiber (Sigma B F) B (pr1 B F) y
:= λ (x : F y), ((y, x), idp B y)

def decode-encode-Pi (B : U) (F : B → U) (y : B) (x : F y)
  : Path (F y) (transp (<i> F (idp B y @ i)) 0 x) x
:= <j> transp (<i> F y) j x

def encode-decode-Pi (B : U) (F : B → U) (y : B)
  (x : fiber (Sigma B F) B (pr1 B F) y)
  : Path (fiber (Sigma B F) B (pr1 B F) y)
    ((y, encode-Pi B F y x), idp B y) x
:= <i> ( (x.2 @ i, transp (<j> F (x.2 @ i ∨ -j)) i x.1.2),
    <j> x.2 @ i ∧ j )

def Bundle=Pι (B : U) (F : B → U) (y : B)

```

```

: PathP (<_> U) (fiber (Sigma B F) B (pr1 B F) y) (F y)
:= iso→Path (fiber (Sigma B F) B (pr1 B F) y) (F y)
  (encode—Pi B F y) (decode—Pi B F y)
  (decode—encode—Pi B F y) (encode—decode—Pi B F y)

```

2.3 Dependent Sum (Σ)

Σ -type is a space that contains dependent pairs where type of the second element depends on the value of the first element. As only one point of fiber domain present in every defined pair, Σ -type is also a dependent sum, where fiber base is a disjoint union.

Σ is a dependent sum type, the generalization of products. Σ type is a total space of fibration. Element of total space is formed as a pair of basepoint and fibration.

Spaces of dependent pairs are using in type theory to model cartesian products, disjoint sums, fiber bundles, vector spaces, telescopes, lenses, contexts, objects, algebras, $\exists$ -type, etc.

Type-theoretical interpretation

Definition 17 (Σ -Formation). The dependent sum type is indexed over type A in the sense of coproduct or disjoint union, where only one fiber codomain $B(x)$ is present in pair.

$$\Sigma(A, B) : U =_{def} \prod_{A:U} \prod_{B:A \rightarrow U} \sum_{x:A} B(x).$$

```

def Sigma (A: U) (B: A → U) : U := Σ (x: A), B(x)

```

Definition 18 (Σ -Introduction). The dependent pair constructor is a way to create indexed pair over type A in the sense of coproduct or disjoint union.

$$\mathbf{pair} : \Sigma(A, B) =_{def} \prod_{A:U} \prod_{B:A \rightarrow U} \prod_{a:A} \prod_{b:B(a)} (a, b).$$

```

def pair (A: U) (B: A → U) (a: A) (b: B a) : Sigma A B := (a, b)

```

Definition 19 (Σ -Elimination). The dependent projections $pr_1 : \Sigma(A, B) \rightarrow A$ and $pr_2 : \prod_{x:\Sigma(A, B)} B(pr_1(x))$ are pair deconstructors.

$$pr_1 : \prod_{A:U} \prod_{B:A \rightarrow U} \prod_{x:\Sigma(A, B)} A =_{def} .1 =_{def} (a, b) \mapsto a.$$

$$pr_2 : \prod_{A:U} \prod_{B:A \rightarrow U} \prod_{x:\Sigma(A, B)} B(x.1) =_{def} .2 =_{def} (a, b) \mapsto b.$$

```

def pr1 (A: U) (B: A → U) (x: Sigma A B) : A := x.1
def pr2 (A: U) (B: A → U) (x: Sigma A B) : B (pr1 A B x) := x.2

```

Definition 20 (Σ -Induction). States that if predicate holds for two projections then predicate holds for total space.

```

def  $\Sigma$ -ind (A : U) (B : A → U)
  (C :  $\Pi$  (s:  $\Sigma$  (x: A), B x), U)
  (g:  $\Pi$  (x: A) (y: B x), C (x,y))
  (p:  $\Sigma$  (x: A), B x)
: C p := g p.1 p.2

```

Theorem 8 (Σ -Computation). $\text{def } \Sigma\text{-}\beta_1$ (A : U) (B : A → U) (a : A) (b : B a) : Path A a (pr1 A B (a , b)) := idp A a

```

def  $\Sigma\text{-}\beta_2$  (A : U) (B : A → U) (a : A) (b : B a)
: Path (B a) b (pr2 A B (a , b)) := idp (B a) b

```

Theorem 9 (Σ -Uniqueness). $\text{def } \Sigma\text{-}\eta$ (A : U) (B : A → U) (p : Sigma A B) : Path (Sigma A B) p (pr1 A B p, pr2 A B p) := idp (Sigma A B) p

Categorical interpretation

Definition 21. (Dependent Sum). The dependent sum along the morphism $f : A \rightarrow B$ in category C is the left adjoint $\Sigma_f : C/A \rightarrow C/B$ of the base change functor.

Set-theoretical interpretation

Theorem 10. (Axiom of Choice). If for all $x : A$ there is $y : B$ such that $R(x, y)$, then there is a function $f : A \rightarrow B$ such that for all $x : A$ there is a witness of $R(x, f(x))$.

```

def ac (A B: U) (R: A → B → U)
  (g:  $\Pi$  (x: A),  $\Sigma$  (y: B), R x y)
:  $\Sigma$  (f: A → B),  $\Pi$  (x: A), R x (f x)
:= ( $\lambda$  (i: A), (g i).1,  $\lambda$  (j: A), (g j).2)

```

Theorem 11. (Total). If fiber over base implies another fiber over the same base then we can construct total space of section over that base with another fiber.

```

def total (A:U) (B C : A → U)
  (f :  $\Pi$  (x:A), B x → C x)
  (w:  $\Sigma$ (x: A), B x)
:  $\Sigma$  (x: A), C x := (w.1, f (w.1) (w.2))

```

2.4 Path Space (Ξ)

The homotopy identity system defines a **Path** space indexed over type A with elements as functions from interval $[0, 1]$ to values of that path space $[0, 1] \rightarrow A$. HoTT book defines two induction principles for identity types: path induction and based path induction.

This ctt file reflects ²CCHM cubicaltt model with connections. For ³ABCFHL yacctt model with variables please refer to ytt file. You may also want to read ⁴BCH, ⁵AFH. There is a ⁶PO paper about CCHM axiomatic in a topos.

Choosing flavour of normal forms for identity system

Here we give brief description of structure inside path spaces:

Bounded Distributive Lattice: A bounded distributive lattice is a type $L : \mathcal{U}$ equipped with binary operations $\wedge : L \rightarrow L \rightarrow L$, $\vee : L \rightarrow L \rightarrow L$, and constants $0 : L$, $1 : L$, satisfying associativity ($a \wedge (b \wedge c) \equiv (a \wedge b) \wedge c$, $a \vee (b \vee c) \equiv (a \vee b) \vee c$), commutativity ($a \wedge b \equiv b \wedge a$, $a \vee b \equiv b \vee a$), idempotence ($a \wedge a \equiv a$, $a \vee a \equiv a$), absorption ($a \wedge (a \vee b) \equiv a$, $a \vee (a \wedge b) \equiv a$), distributivity ($a \wedge (b \vee c) \equiv (a \wedge b) \vee (a \wedge c)$, $a \vee (b \wedge c) \equiv (a \vee b) \wedge (a \vee c)$), and bounds ($a \wedge 0 \equiv 0$, $a \vee 1 \equiv 1$). In a Boolean topos, L corresponds to the type of subobjects with $\wedge \equiv \times$, $\vee \equiv +$, $0 \equiv \perp$, $1 \equiv \top$.

De Morgan Algebra: A De Morgan Algebra in HoTT is a bounded distributive lattice $(L, \wedge, \vee, 0, 1) : \mathcal{U}$ equipped with a unary operation $\neg : L \rightarrow L$ satisfying De Morgan's Laws ($\neg(a \wedge b) \equiv \neg a \vee \neg b$, $\neg(a \vee b) \equiv \neg a \wedge \neg b$) and involution ($\neg\neg a \equiv a$). The type L models propositions with a negation operation preserving these equivalences, and in a Boolean topos, $L \cong 2 = \{\text{true}, \text{false}\}$ forms a Boolean algebra, satisfying De Morgan's Laws as isomorphisms.

Heyting Algebra: A Heyting Algebra in HoTT is a bounded distributive lattice $(L, \wedge, \vee, 0, 1) : \mathcal{U}$ equipped with an implication operation $\rightarrow : L \rightarrow L \rightarrow L$ such that, for all $a, b, c : L$, there is an equivalence $a \leq b \rightarrow c \iff a \wedge b \leq c$, where $\leq$ is the partial order defined by $a \leq b \iff a \wedge b \equiv a$. Negation is defined as $\neg a \equiv a \rightarrow 0$, and modus ponens holds: given $a : A$ and $f : A \rightarrow B$, there exists $f a : B$. In a Boolean topos, the Heyting algebra becomes a Boolean algebra, with $\rightarrow$ corresponding to the exponential B^A .

Boolean Algebra: A Boolean Algebra in HoTT is a De Morgan Algebra $(L, \wedge, \vee, \neg, 0, 1) : \mathcal{U}$ satisfying the law of excluded middle ($a \vee \neg a \equiv 1$) and non-contradiction ($a \wedge \neg a \equiv 0$). The type $L \cong 2 = \{\text{true}, \text{false}\}$ models classical

²Cyril Cohen, Thierry Coquand, Simon Huber, Anders Mörtberg. Cubical Type Theory: a constructive interpretation of the univalence axiom. 2015. <https://5ht.co/cubicaltt.pdf>

³Carlo Angiuli, Brunerie, Coquand, Kuen-Bang Hou (Favonia), Robert Harper, Dan Licata. Cartesian Cubical Type Theory. 2017. <https://5ht.co/ccctt.pdf>

⁴Marc Bezem, Thierry Coquand, Simon Huber. A model of type theory in cubical sets. 2014. <http://www.cse.chalmers.se/~coquand/mod1.pdf>

⁵Carlo Angiuli, Kuen-Bang Hou (Favonia), Robert Harper. Cartesian Cubical Computational Type Theory: Constructive Reasoning with Paths and Equalities. 2018. <https://www.cs.cmu.edu/~cangiuli/papers/ccctt.pdf>

⁶Andrew Pitts, Ian Orton. Axioms for Modelling Cubical Type Theory in a Topos. 2016. <https://arxiv.org/pdf/1712.04864.pdf>

propositions, with a mandatory Boolean type in a Boolean topos, where L is the subobject classifier $\Omega \cong 2$, and all operations correspond to classical logical connectives.

In **Per** De Morgan algebra is used (CCHM flavour).

Type-theoretical interpretation

Definition 22 (Path Formation).

$$\Xi(A, x, y) : U =_{def} \prod_{A:U} \prod_{x,y:A} \mathbf{Path}_A(x, y).$$

```
def Path (A : U) (x y : A) : U
:= PathP (<-> A) x y

def Path' (A : U) (x y : A)
:=  $\Pi (i : I), A [\partial \ i \ |-> [(i = 0) \rightarrow x, (i = 1) \rightarrow y ]]$ 
```

Definition 23 (Path Introduction).

$$\mathbf{idp} : x \equiv_A x =_{def} \prod_{A:U} \prod_{x:A} [i]x.$$

```
def idp (A: U) (x: A) : Path A x x := <-> x
```

Returns a reflexivity path space for a given value of the type. The inhabitant of that path space is the lambda on the homotopy interval $[0, 1]$ that returns a constant value x . Written in syntax as $[i]x$.

Definition 24 (Path Application). You can apply face to path.

```
def at0 (A: U) (a b : A) (p: Path A a b) : A := p @ 0
def at1 (A: U) (a b : A) (p: Path A a b) : A := p @ 1
```

Definition 25 (Path Composition). Composition operation allows to build a new path by given to paths in a connected point.

$$\begin{array}{ccc} & a & \xrightarrow{comp} c \\ \lambda(i : I) \rightarrow a \uparrow & & \uparrow q \\ & a & \xrightarrow{p @ i} b \end{array}$$

```
def pcomp (A : U) (a b c : A) (p : Path A a b) (q : Path A b c)
: Path A a c
:= <i> hcomp A ( $\partial \ i$ ) ( $\lambda (j : I), [(i = 0) \rightarrow a,$ 
 $(i = 1) \rightarrow q @ j]$ ) (p @ i)
```

Theorem 12 (Path Inversion).

```
def inv (A: U) (a b: A) (p: Path A a b) : Path A b a := <i> p @ -i
```

Definition 26 (Connections). Connections allows you to build square with given only one element of path: i) $\lambda (i, j : I) \rightarrow p @ \min(i, j)$; ii) $\lambda (i, j : I) \rightarrow p @ \max(i, j)$.

$$\begin{array}{ccc}
 & a & \xrightarrow{p} & b \\
 \lambda (i : I) \rightarrow a \uparrow & & & \uparrow p \\
 a & \xrightarrow{\lambda (i : I) \rightarrow a} & a & \\
 & & & \\
 & b & \xrightarrow{\lambda (i : I) \rightarrow b} & b \\
 p \uparrow & & & \uparrow \lambda (i : I) \rightarrow b \\
 a & \xrightarrow{p} & b &
 \end{array}$$

```
def meet (A: U) (a b: A) (p: Path A a b)
  : PathP (<x> Path A a (p@x)) (<i>a) p
  = <x y> p @ (x /\ y)
```

```
def join (A: U) (a b: A) (p: Path A a b)
  : PathP (<x> Path A (p@x) b) p (<i>b)
  = <y x> p @ (x /\ y)
```

Theorem 13 (Congruence). Is a map between values of one type to path space of another type by an encode function between types. Implemented as lambda defined on $[0, 1]$ that returns application of encode function to path application of the given path to lamda argument $\lambda(i : I), f(p@i)$ for both cases.

$$\text{ap} : f(a) \equiv f(b) =_{def}$$

$$\prod_{A:U} \prod_{a,x:A} \prod_{B:A \rightarrow U} \prod_{f:\Pi(A,B)} \prod_{p:a \equiv_A x} [i]f(p@i).$$

```
def ap (A B: U) (f: A -> B) (a b: A) (p: Path A a b)
  : Path B (f a) (f b)
```

```
def apd (A: U) (a x: A) (B: A -> U)
  (f: A -> B a) (b: B a) (p: Path A a x)
  : Path (B a) (f a) (f x)
```

Theorem 14 (Generalized Transport Kan Operation). Transports a value of the left type to the value of the right type by a given path element of the path space between left and right types.

$$\text{transport} : A(0) \rightarrow A(1) =_{def}$$

$$\prod_{A:I \rightarrow U} \prod_{r:I} \lambda x, \text{transp}([i]A(i), 0, x).$$

```

def transp' (A: U) (x y: A) (p : PathP (<->A) x y) (i: I)
:= transp (<i> (\(-:A),A) (p @ i)) i x

def transp-U (A B: U) (p : PathP (<->U) A B) (i: I)
:= transp (<i> (\(-:U),U) (p @ i)) i A

```

Definition 27 (Singleton). $\text{def } \text{singl } (A: U) (a: A): U := \Sigma (x: A), \Xi$
 $A \ a \ x$

Theorem 15 (Singleton Instance). $\text{def } \text{eta } (A: U) (a: A): \text{singl } A \ a := (a, \text{idp } A \ a)$

Theorem 16 (Singleton Contractability). $\text{def } \text{contr } (A : U) (a \ b : A) (p : \Xi$
 $A \ a \ b)$
 $: \Xi (\text{singl } A \ a) (\text{eta } A \ a) (b, p)$
 $:= <i> (p @ i, <j> p @ i /\ j)$

Theorem 17 (Path Elimination). $\text{def } \text{subst } (A : U) (P : A \rightarrow U) (a \ b : A)$
 $(p : \Xi A \ a \ b) (e : P \ a) : P \ b$
 $:= \text{transp } (<i> P (p @ i)) \ 0 \ e$

```

def D (A : U) : U1
:=  $\Pi (x \ y : A), \text{Path } A \ x \ y \rightarrow U$ 

```

```

def J (A: U) (x: A) (C: D A) (d: C x x (idp A x))
(y: A) (p:  $\Xi A \ x \ y$ ) : C x y p
:= subst (singl A x) (\ (z: singl A x), C x (z.1) (z.2))
(eta A x) (y, p) (contr A x y p) d

```

Theorem 18. (Path Computation).

```

def trans_comp (A : U) (a : A)
:  $\Xi A \ a (\text{transport } A \ A (<i> A) \ a)$ 
:= <j> transp (<-> A) -j a

def subst-comp (A: U) (P: A → U) (a: A) (e: P a)
:  $\Xi (P \ a) \ e (\text{subst } A \ P \ a \ a (\text{idp } A \ a) \ e)$ 
:= trans_comp (P a) e

def J-β (A : U) (a : A) (C : D A) (d: C a a (idp A a))
:  $\Xi (C \ a \ a (\text{idp } A \ a)) \ d (J \ A \ a \ C \ d \ a (\text{idp } A \ a))$ 
:= subst-comp (singl A a)
(\ (z: singl A a), C a (z.1) (z.2)) (eta A a) d

```

Note that Path type has no Eta rule due to groupoid interpretation.

Groupoid interpretation

The groupoid interpretation of type theory is well known article by Martin Hofmann and Thomas Streicher, more specific interpretation of identity type as infinity groupoid [6].

Contexts

In Martin-Löf Type Theory (MLTT), contexts define the typing environment for judgments, consisting of a sequence of typed variable declarations that enable the derivation of types and terms.

Context as metatheoretical entity couldn't be internalized but could be imagined as telescopes, ensuring well-formedness and supporting constructive type checking. Explicit context rendering could be seen in categorical interpretation of dependent type theory

Definition 28 (Empty Context). The empty context contains no variable declarations and serves as the base case for context formation. It is represented as the unit type, indicating an empty telescope:

$$\gamma_0 : \Gamma =_{def} \star.$$

Definition 29 (Context Comprehension). A context is extended by adding a variable declaration for a type dependent on the existing context. For a context Γ and a type A over Γ , the extended context is:

$$\Gamma; A =_{def} \sum_{\gamma:\Gamma} A(\gamma).$$

This is encoded as a dependent pair, binding a variable to a type in the context.

Definition 30 (Context Derivability). A type A is derivable in a context Γ if it can be assigned to a universe given the variables in Γ :

$$\Gamma \vdash A =_{def} \prod_{\gamma:\Gamma} A(\gamma).$$

This corresponds to a dependent function type, ensuring A is well-typed across all context elements: For terms, a term $t : A$ in Γ , written $\Gamma \vdash t : A$, is derivable if it respects the context's bindings.

Definition 31 (Terms). A term is an element of a type within a context. Given $\Gamma \vdash A : U_i$, a term t satisfies $\Gamma \vdash t : A$. Terms include variables, constructors (e.g., λ for Π , pairs for Σ), and applications, defined by MLTT-73's syntax.

Contexts provide a structured environment for deriving judgments. They integrate with the any reasoning framework, supporting and ensuring sequential constructive verification.

MLTT-73

Here is given formal model of type-theoretical interpretation of Martin-Löf Type Theory. It combines 4 Path rules (no eta), 5 Π rules, and 6 Σ rules (two elims). The proof is provided by direct embedding (internalizing) the model into the model of type checker which is even more powerful.

Definition 32 (MLTT-73 Reality Check). The MLTT as a Type is defined by taking all rules for Π , Σ and Path types into one Σ telescope or context.

```
def MLTT-73 (A : U) : U1 :=
  Σ (Π-form : Π (B : A → U), U)
    (Π-ctor1 : Π (B : A → U), Π A B → Π A B)
    (Π-elim1 : Π (B : A → U), Π A B → Π A B)
    (Π-comp1 : Π (B : A → U) (a : A) (f : Π A B),
      ≡ (≡ (B a) (Π-elim1 B (Π-ctor1 B f) a) (f a)))
    (Π-comp2 : Π (B : A → U) (a : A) (f : Π A B),
      ≡ (Π A B) f (λ (x : A), f x))
    (Σ-form : Π (B : A → U), U)
    (Σ-ctor1 : Π (B : A → U) (a : A) (b : B a), Sigma A B)
    (Σ-elim1 : Π (B : A → U) (p : Sigma A B), A)
    (Σ-elim2 : Π (B : A → U) (p : Sigma A B), B (pr1 A B p))
    (Σ-comp1 : Π (B : A → U) (a : A) (b : B a),
      ≡ A a (Σ-elim1 B (Σ-ctor1 B a b)))
    (Σ-comp2 : Π (B : A → U) (a : A) (b : B a),
      ≡ (B a) b (Σ-elim2 B (a, b)))
    (Σ-comp3 : Π (B : A → U) (p : Sigma A B),
      ≡ (Sigma A B) p (pr1 A B p, pr2 A B p))
    (=form : Π (a : A), A → U)
    (=ctor1 : Π (a : A), Path A a a)
    (=elim1 : Π (a : A) (C : D A) (d : C a a (=ctor1 a))
      (y : A) (p : Path A a y), C a y p)
    (=comp1 : Π (a : A) (C : D A) (d : C a a (=ctor1 a)),
      ≡ (C a a (=ctor1 a)) d
        (=elim1 a C d a (=ctor1 a))), 1
```

Theorem 19. (Model Check). There is an instance of MLTT.

```
def internalizing (A : U) : MLTT A
:= ( Π A, Π-lambda A, Π-apply A, Π-β A, Π-η A,
    Sigma A, pair A, pr1 A, pr2 A, Σ-β1 A, Σ-β2 A, Σ-η A,
    Path A, idp A, J A, J-β A, ★ )
```

The result of the work is a `mltt.ctt` file which can be runned using `cubicaltt`. Note that MLTT-73 internalization includes only eliminator and computational rule for identity system (without uniqueness rule), as cubical Path spaces refute uniqueness of identity proofs.

Conclusions

This article presents a landmark achievement in type theory: the constructive internalization of Martin-Löf Type Theory (MLTT-73) computational rules within the **Per** language, a minimal type system equipped with cubical type theory primitives.

This internalization, formalized also in the `mltt.ctt` for double checking, validates MLTT-73 in `cubicaltt`, providing a rigorous test of a type checker's ability to fuse introduction and elimination rules through computational and uniqueness equations.

The significance of this work lies in its constructive approach to the J eliminator, a cornerstone of MLTT-73 identity type, which previous internalization

Language	U^n	Π	Σ	Id	Ξ	$\mathbb{N}$	0/1/2	W	Ind	Fix
System P_ω (CoC-88)		x								
MLTT-72		x	x							
Henk (ECC)	x	x								
Errett (LCCC/IPL)	x	x	x				x			
MLTT-73	x	x	x	x						
Per	x	x	x	x	x		x	x		
MLTT-75	x	x	x	x		x	x			
MLTT-80	x	x	x	x				x		
Anders (HTS)	x	x	x	x	x	x	x	x		
Frank (CoC+CIC)	x	x							x	
Christine (Coq)	x	x							x	x
cubicaltt		x	x		x				x	x
Agda	x	x	x	x	x				x	x
Lean	x	x	x	x					x	x
NuPRL		x	x	x					x	

attempts failed to derive constructively [3, 10]. By leveraging cubical type theory’s Path types and operations (e.g., connections, compositions), the type checker achieves a compact foundational core for verifying mathematics.

The article also elucidates MLTT-73 versatility through logical, categorical, homotopical, and set-theoretical interpretations, offering a comprehensive landscape for researchers and newcomers to type theory.

References

- [1] Vladimir Voevodsky et al., *Homotopy Type Theory*, in *Univalent Foundations of Mathematics*, 2013.
- [2] Per Martin-Löf and Giovanni Sambin, *The Theory of Types*, in *Studies in Proof Theory*, 1972.
- [3] Per Martin-Löf, *An Intuitionistic Theory of Types: Predicative Part*, in *Studies in Logic and the Foundations of Mathematics*, vol. 80, pp. 73–118, 1975. doi:10.1016/S0049-237X(08)71945-1
- [4] Per Martin-Löf and Giovanni Sambin, *Intuitionistic Type Theory*, in *Studies in Proof Theory*, 1984.
- [5] Thierry Coquand and Gérard Huet, *The Calculus of Constructions*, in *Information and Computation*, pp. 95–120, 1988. doi:10.1016/0890-5401(88)90005-3
- [6] Martin Hofmann and Thomas Streicher, *The Groupoid Interpretation of Type Theory*, in *Venice Festschrift*, Oxford University Press, pp. 83–111, 1996.

- [7] Claudio Hermida and Bart Jacobs, *Fibrations with Indeterminates: Contextual and Functional Completeness for Polymorphic Lambda Calculi*, in *Mathematical Structures in Computer Science*, vol. 5, pp. 501–531, 1995.
- [8] Alexandre Buisse and Peter Dybjer, *The Interpretation of Intuitionistic Type Theory in Locally Cartesian Closed Categories – an Intuitionistic Perspective*, in *Electronic Notes in Theoretical Computer Science*, pp. 21–32, 2008. doi:10.1016/j.entcs.2008.10.003
- [9] Errett Bishop, *Foundations of Constructive Analysis*, 1967.
- [10] Bengt Nordström, Kent Petersson, and Jan M. Smith, *Programming in Martin-Löf’s Type Theory*, Oxford University Press, 1990.
- [11] Matthieu Sozeau and Nicolas Tabareau, *Internalizing Intensional Type Theory*, unpublished.
- [12] Martin Hofmann and Thomas Streicher, *The Groupoid Model Refutes Uniqueness of Identity Proofs*, in *Logic in Computer Science (LICS’94)*, IEEE, pp. 208–212, 1994.
- [13] Bart Jacobs, *Categorical Logic and Type Theory*, vol. 141, 1999.
- [14] Anders Mörtberg et al., *Cubical Type Theory: A Constructive Interpretation of the Univalence Axiom*, arXiv:1611.02108, 2017.
- [15] Simon Huber, *Cubical Interpretations of Type Theory*, Ph.D. thesis, Dept. of Computer Science and Engineering, University of Gothenburg, 2016.
- [16] Maksym Sokhatskyi and Pavlo Maslianko, *The Systems Engineering of Consistent Pure Language with Effect Type System for Certified Applications and Higher Languages*, in *Proc. 4th Int. Conf. Mathematical Models and Computational Techniques in Science and Engineering*, 2018. doi:10.1063/1.5045439